

Responsibility-Driven Architecture

Stuart Blair and Richard Watt, *Outformations*

Tim Cull, *TheDewick*

An implementation of real options theory informs when, how, and who should make architectural decisions and reframes the architect's role in agile development.

The architect's role in agile development remains at best ill-defined and at worst simply ignored. On one hand, our experiences with agile show that there's little value in trying to anticipate future needs with incomplete information. On the other, it's dangerous to make decisions in a purely emergent fashion. This led us to seek an alternative that falls somewhere between these approaches.

Responsibility-driven architecture (RDA) explores when, how, and who should make architectural decisions. Our research attempts to answer these questions from an agile perspective and proposes a framework to improve architectural design. We focus more on the most responsible moment to make architectural decisions and how best to make them. We also identify different architectural stakeholders' roles to ensure everyone makes the most useful contributions possible.

We found this approach especially helpful in bridging agile development teams with decidedly nonagile enterprises and relatively strict, waterfall-inspired governance processes. Our approach gives organizations independent architectural review and oversight from outside the project team, without weighing down the team's agile methodology. We think of it less as enterprise or application architecture, and more as essential architecture.

Balancing Emergent and Up-Front Design

When weighing responsibility driven architecture against big design up-front or emergent architectural design, we lean more towards the latter but are conscious of some value in the former, espe-

cially for larger architectural concerns. We reframed the question in terms of when was the earliest moment we could reasonably begin to think about a particular architectural aspect, and when was the latest responsible moment that we should have made a decision.

Theory

The traditional model for defining a software system's architecture is to identify constraints and standards in the early stages of development and then apply them to the system's requirements. This approach—commonly referred to as big design up-front (BDUF)—assumes that the project's beginning is the best time to make architectural decisions. However, it's impossible to predict all the constraints and standards that should be applied at such an early stage. So, when is the right time?

In direct reaction to the BDUF philosophy, emergent design¹ is an attempt from the agile community, and the Extreme Programming (XP) movement in particular, to evolve the architecture directly through the act of programming and the practice of test-driven development. Emergent design argues that architecture is no different from any other design quality and can be developed in

step with the changing business requirements, thereby flattening the cost of change anticipated by BDUF. Coupled with this is a belief that we should avoid adding any structural element that isn't driven by a business need, or in XP parlance, "you ain't gonna need it" (YAGNI). This belief has led many agile teams ultimately to a point of failure by ignoring the architecture's other essential elements. Even in successful cases, teams fail to realize the full promise of emergent design because only thinking about what the customer needs next doesn't necessarily provide the foundation to successfully maintain a productive environment and avoid the descent into software entropy. But what are the essential elements and when should project teams make those decisions?

Unlike BDUF, emergent design assumes that developers can make architectural decisions on an ongoing basis as part of the natural development cycle. In exploring this topic, we discovered the potential for using real options theory² to help teams split the difference and decide on the best time to make architectural decisions. While many agile teams claim they do no up-front design, research by Scott Ambler suggests that they do.³ Real options in general, and our approach in particular, give them a way to frame their up-front work.

Real options is a tool used in corporate finance to assist in investment decisions. It represents the right—but not the obligation—to make a particular business decision by a particular time known as the expiry, after which the right is no longer available. For example, a business might model whether to develop a feature for a new product to catch the Christmas market as a real option. Rather than committing to provide the feature as soon as it's suggested, the option lets business defer the decision to a later point. Beyond that expiry point, however, it will be too late for the Christmas market. By deferring the decision, the business can use its resources to concentrate on the issues that it genuinely needs to address. More importantly, in the time between the option's inception and its expiry, the business can benefit by actively exploring new options that couldn't have been foreseen at the beginning of the product cycle. In addition, the business can abandon the right to develop the feature if marketplace changes or intelligence gathered in the interim render it undesirable.

Project teams can apply the theory of real options in a different context. In the case of architecture, it lets project teams reframe the decision-making process, resulting in significant benefits. That's not to say that making some architectural decisions up front is wrong—quite the reverse. It

also allows for some decisions that can't be deferred, which is often still the best choice. The key is knowing the most appropriate time to make a decision. Or expressed differently, when is the latest responsible moment to make a decision? By drawing on real options theory, we've developed a process that allows development teams and contributing architects to collaborate on decision-making and thus realize the benefits of an architectural design that fits more closely with the larger enterprise's needs and standards.

Implementation

We applied real options using an architecture framework developed by Nick Rozanski and Eóin Woods⁴ but, as we noted in the introduction, the process is intentionally agnostic and can accommodate any architectural framework. Our implementation employed a simple spreadsheet design (see Figure 1).

First, the spreadsheet maps architectural concerns (y-axis) to development phases (x-axis). For example, the design of the security aspect in Figure 1 starts in the business requirements phase and ends in the technical design phase. So, the earliest point that we'd expect the development team to start considering security aspects is in the business requirements phase, and the latest we'd expect a decision is by the end of the technical design phase.

Second, in working with the team, we started with a default template and adjusted it as necessary. In the real-life example depicted in Figure 1, our flexible template let us adjust the generic, waterfall-inspired milestones our enterprise governance process dictated to fit an agile project that broke its ultimate delivery into four, quarter-long releases. The key is to delay the decision to as late as is useful, (but no later) to make the most informed decision. Some teams are tempted to make decisions as early as possible (BDUF), but they need to understand that postponing a decision lets them learn more about the problem space and potentially generate more options and ideas.

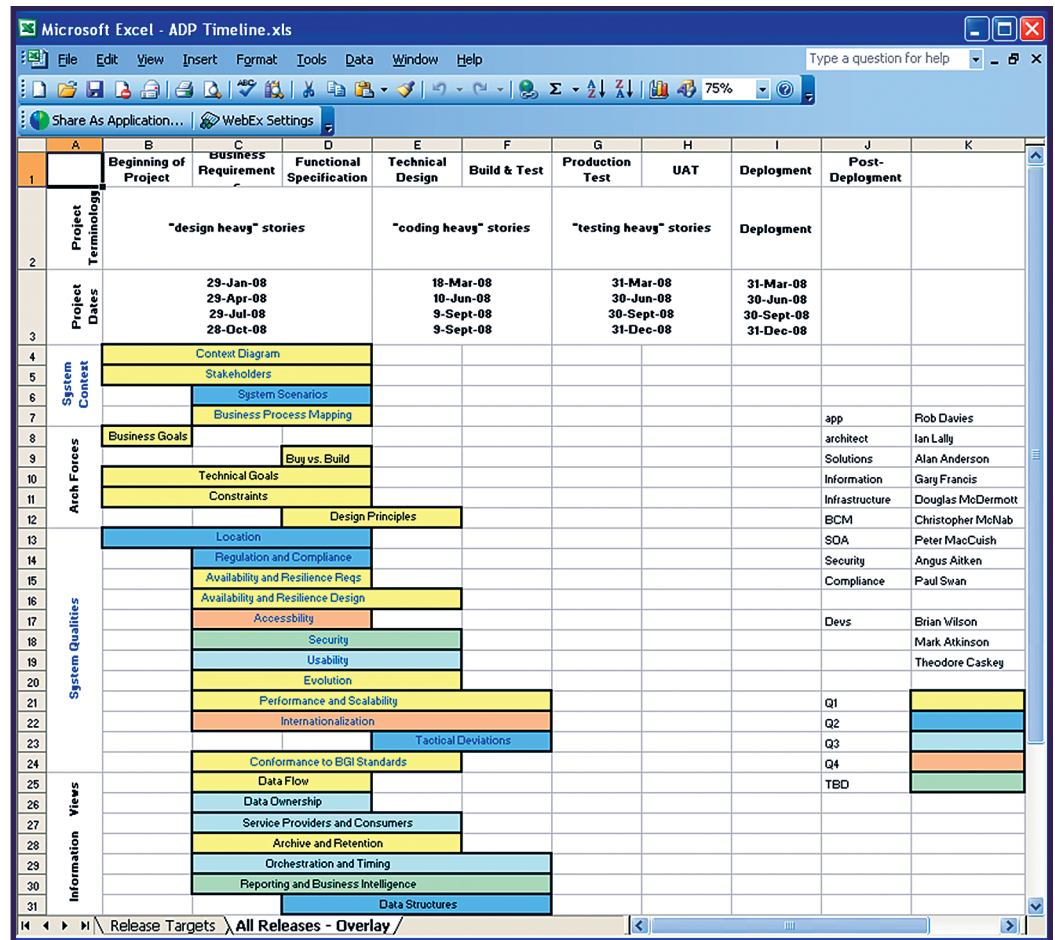
Finally, the spreadsheet acts as a roadmap to show the other stakeholders when they should expect to engage with the development team. When applied at our company, it allowed some of the more overworked central groups (operations and systems admin groups) to see changes coming earlier, rather than being surprised when the change ticket for their release was published.

Architect as Facilitator

The second facet of our architectural approach involves looking at architecture as a service rather

The key is knowing the most appropriate time to make a decision.

Figure 1. Architectural development process (ADP) timeline spreadsheet. The spreadsheet maps four agile releases for one real-life project phase onto the enterprise's waterfall-inspired governance process.



than an imposition. Our aim was to reverse the role and position of the traditional architecture group and present ourselves as service providers—to help teams do the “right thing” from an organizational perspective—instead of lofty architects making unreasonable demands on the backs of development teams under pressure. The implementation pattern we used to realize this idea we call *architect as facilitator*. It’s our job to do whatever work is necessary to ensure the development teams are building architecturally sound applications, and to do so in a collaborative, hands-on way.

Theory

When architects approach their jobs with a servant-leader mindset rather than a command-and-control mindset, there’s a subtle but important change in the way the development team interacts with the rest of the enterprise. We call this architect as facilitator over *architect as façade*.

This concept might sound similar to an *architecture owner*,⁶ but the two concepts describe solutions to different problems: architecture owner primarily describes how a development team works inwardly. Architecture as facilitator pri-

marily describes how a development team works outwardly with the rest of the (likely nonagile) enterprise.

Traditional architecture has the architect working as a façade to the rest of the organization. Stakeholders such as security and infrastructure groups feed their requirements to the architect, who synthesizes them into designs and policies and feeds them to development teams. If these teams have a problem with the designs or policies, they take it up with the architect. If stakeholders believe that applications aren’t being designed to meet their concerns, they also take it up with the architect. This has four major drawbacks:

- The architect becomes a bottleneck in the information pipeline, and the organization loses efficiency.
- Information is lost—in particular, the true intention behind stakeholder requirements is vulnerable to the architect’s interpretation and can get distorted.
- The feedback cycle is broken. Stakeholders are left to develop policies in a vacuum without knowing what’s relevant or the true cost of

their decisions to the rest of the organization.

- Accountability is blurred. If stakeholders write unimplementable policies, whose fault is it? If applications aren't designed to meet stakeholder needs, whose fault is it? Development teams and stakeholders could easily blame the architect, but what real empowerment does the architect have to fix the problem if that person didn't write the policies or the code?

We propose a shift in the interaction model that fixes all four problems. Figure 2 shows the interaction with architect as façade.

Notice in Figure 2 how pinched the communication channel is through the architect and the potential to lose information. Now compare that diagram to the model we call architect as facilitator in Figure 3. There, the architect unambiguously defines who's accountable for policy and who the development teams should contact for approval and consultation. The architect then introduces both parties and facilitates conversations between them to clear up any misunderstandings. Critically, stakeholders talk directly to developers and both are clear that they're accountable to each other without needing an architect in the middle to blur that accountability.

Implementation

It's easier to describe the architect as facilitator than it is to implement this model, mostly because the shift appears subtle on the surface. Through our work with the ADP, we've found that it's critical to spell out this approach explicitly and repeatedly with all stakeholders and development teams. An excellent tool to make sure everyone's responsibilities are clear is a RACI matrix.⁵ Briefly, RACI stands for responsible, accountable, consulted, and informed. Each aspect describes a person's role in designing the application and the nature of the interaction between them and everyone else.

In most cases, the development team is the responsible party—the people who physically do the work of designing and coding the application. The facilitator architect then ensures exactly one person per area of architectural concern (that is, availability, deployability, auditability, and so on) is accountable, which means they're responsible for approving or signing off on the result. Consulted parties are stakeholders who expect to give feedback to the project to influence its design and also receive information from the project (two-way communication). Informed parties are stakeholders who just receive information from the project but generally don't give feedback to influence its

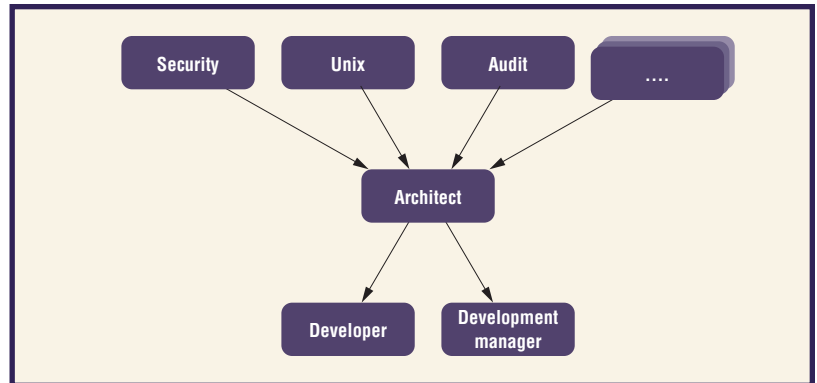


Figure 2. Architect as façade. The architect as façade works between developers and stakeholders, constricting lines of communication.

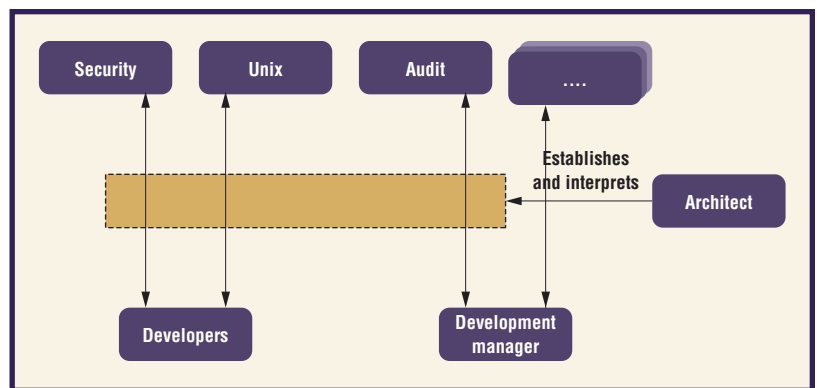


Figure 3. Architect as facilitator. The architect as facilitator works among developers and stakeholders, allowing more direct interaction.

design (one-way communication). Unlike with the accountable party, there could be more than one consulted and more than one informed party per area of architectural concern.

The accountable party is typically the person whose job it is to worry about that particular area of concern. For example, the head of information security (or someone who works for that person) would be the accountable party for the security aspects of the application's design. It's also entirely appropriate for the development team to be both responsible and accountable at the same time; for example, it might be responsible and accountable for the source code build process and structure.

The trick to successfully implementing a RACI matrix for architectural design is to clearly define a few critical things:

- What are the different areas of architectural concern (for example, security and availability), and what are their boundaries? Make sure they don't overlap.
- Which individuals occupy each role—for example, security, Unix administrators, and audit?

Our implementation started with a list of concerns that largely mirrored Rozanski and Woods'

Concern	Security	Unix	Audit	Developers
Availability	C	I		RA
Deployability	I	A		R
Auditability	C		A	R

Figure 4. RACI (responsible, accountable, consulted and informed) matrix. The matrix unambiguously assigns architectural perspectives (along the left edge) to different roles in the organization (along the top). Each letter represents a different aspect of RACI.

perspectives and a list of stakeholders that largely mirrored Rozanski and Woods' viewpoints,⁴ and then customized them to fit our organization. For example, in our implementation, we split the concept of availability into two concerns: requirements and design. Our company's business continuity management (BCM) team was accountable for defining the availability requirements for different kinds of applications—for example, a 30-minute recovery time objective for applications that handled money. Our company's infrastructure team was accountable for signing off that the proposed design was actually capable of meeting those requirements—for example, the Sybase replication server to a warm-standby database could be failed over in less than 30 minutes. We found that it's okay to mix and match roles, provided that there's clear accountability—that is, what needs to be done, by whom, and when.

Once the architect has clearly defined these roles, he or she can construct a matrix that resembles the one in Figure 4.

Architecture Development Process

The Architectural Development Process (ADP) is our implementation of the responsibility driven-architecture approach. The ADP is a collaborative process for improving systems architecture and is designed to ensure that all architectural concerns are addressed at the appropriate time. The tool was developed to help ensure application teams are given the necessary information and support they need to build enterprise-worthy applications. The process is intentionally agnostic and can accommodate any architectural framework.

Theory

Agile development teams often feel a tension between their desire for flexibility and autonomy and the desire of external parties—for example, regulators and auditors—to verify that software was developed with due process. Although this tension is common, it isn't strictly necessary.

It isn't difficult to find auditors who've experienced software teams cutting corners and ignoring nonfunctional requirements, leading to dire con-

sequences when the software reaches production. It's reasonable then to insist on documentation at each stage of the development process, signed off by several independent parties. However, this high-ceremony solution is incongruent with agile teams' sensibilities. Agile teams want the flexibility to change the software in-flight as requirements emerge. Because they see the software development as continuous and evolutionary, the notion of up-front sign-off of the architecture only seems to impede development by forcing them to make architectural decisions in a vacuum of experience. The opportunity to make better architectural decisions is lost because those decisions can't be informed by the growing body of experience as the project progresses.

Both parties have legitimate concerns, but there's a way to address them if everyone is willing to be flexible and compromise.

Implementation

Really, the auditor wants to see that important parts of the architecture were thought through and that the team consulted the correct authorities within the organization. Really, the development team wants to focus on the right thing at the right time, not a minute sooner or later. Both sides want to make sure nothing important is missed—and this common thread can eventually bring them together.

Contrary to popular belief, agile teams do perform some up-front planning,³ it's just that they don't think of it that way. To get funding for a project in the real corporate world, someone has to put together a high-level description of what the software will do, its approximate boundaries, and roughly how much it will cost to build. Then, once the project is funded, the development team is assembled and starts planning releases. Releases, in turn, contain iterations. Typically, there's some kind of "iteration 0" period when the team comes up with its ground rules and gets its head around what it signed up to do. This period can be leveraged to make stakeholders and the auditor comfortable that their concerns will be met before the organization incurs any risk. The auditor will get the same sign-off as with traditional projects, but in a progressive manner.

Our implementation of progressive sign-off is part of the ADP, which takes the traditional inception-design-code-test-deploy milestones and their associated paperwork, chops them into smaller pieces, and allocates them to places in the project that make sense. By the end of the project, the auditor gets the paperwork he or she needs to feel

comfortable, but that paperwork was developed progressively.

This progressive sign-off is really nothing more than adding a dash of ceremony to the typical agile design process. One of the fundamental assumptions of agile development is that it's impossible to know the full scope of development at the project's beginning. Nevertheless, the team does know up front that there are certain themes it will have to tackle eventually, such as resilience, recovery, audit trail, performance, deployment, and so on. Without knowing what exactly will be done, it's possible to say something like, "We'll get our deployment situation nailed by the last iteration of release 1, and most likely will have it 80 percent sorted by iteration 3." It's also possible for the team to say something like, "Whatever we develop, it will keep our Unix team happy and will satisfy the deployment requirements it published on its wiki."

In implementing continuous development, we followed these steps:

- **Find stakeholders.** The team assembles descriptions of all the stakeholder (that is, Unix team, security folks, and auditors) requirements from around the firm. If the team is lucky and works in our organization, then the enterprise architecture team will have done that for them and put it all in one place. If the team is unlucky, it will have to scour wikis and shared folders looking for requirements. The RACI matrix we described earlier points the team to the right stakeholders. In our implementation (the ADP), the enterprise architecture team gathered requirements from stakeholders across the enterprise and rephrased them into guiding questions the development teams could use to spark their design process.
- **Commit to stakeholders.** The team commits to meeting the requirements but makes clear to its stakeholders—that is, the Unix team, security folks, and auditors—that although it doesn't know how it will meet the requirements, it will indeed meet them to stakeholder satisfaction and keep stakeholders informed of progress.
- **Map requirements to implementation.** Now the team takes those requirements and groups them into buckets that it assumes make sense to implement together—that is, "availability and resiliency," "functional structure," "performance and scalability." Again, if the team is lucky, the architects have already done this for them. If the team is less fortunate, they must do it themselves.
- **Just-in-time document work.** Finally, the team

maps those buckets onto planned releases. As a team, it commits to meeting each bucket's requirements by the end of the release and documenting how it met them. When the team and its customers jointly sign up for stories at the beginning of each iteration, they keep in mind that they've also committed to stakeholders to meet the requirements in that release's bucket.

Using the ADP, we can show verifiable sign-off (that is, for regulators and auditors) without tying project teams down to decisions sooner than they need to be made. At the end of the project, all documentation is comprehensive, but it was created in real-time, incrementally.

The worlds of agile development and enterprise architecture don't have to be mutually exclusive. We've presented what we call responsibility driven architecture—reusing the term⁵ but expanding on its meaning—to bring these worlds closer together.

Our framework helps diffuse some of the tension agile teams often feel in process-heavy companies, while still meeting the regulatory requirements many organizations face. Also, it can help scale agile development into realms where it's historically had difficulty and help agile teams navigate cross-cutting design decisions without reverting to a waterfall paradigm.

When we implemented this approach, we replaced an existing architectural review process that had failed largely because it was a point-in-time questionnaire completed by teams either too early or too late in the development process and was viewed by the team as a bureaucratic inconvenience that added little value. In comparison, our process provided an engagement model that encouraged architects and developers to collaborate early and often, continuously sharing ideas.

Of course, no implementation passes without its challenges. We were surprised to discover that some of our architectural stakeholders needed to be nudged to participate directly with development teams. Sometimes, they appeared as uncomfortable talking directly to developers as developers were talking to them.

We've described an engagement model where architects become servant leaders. Not all are comfortable with the notion of servant leadership, but if architecture is to be more than just a collection of mostly ignored policy documents and checklists, then there's no replacement for active participation and the responsibility that it brings. ☞

Not all are comfortable with the notion of servant leadership.

About the Authors



Stuart Blair is an application architect with experience in delivering enterprise applications for buy-side and sell-side firms in the finance industry. His interests include applying agile methods to the development of both large-scale software systems and the teams that build them. Blair has a PhD in computer science from the University of Glasgow. Contact him at stuart@5values.com.

Richard Watt is a director of project portfolio management working in the asset management industry. His interests include the application of agile principles to architecture and technical design. He's an active practitioner in the agile community and has been a regular presenter at international agile conferences since 2004. Watt has an MSc in human-computer interaction from Heriot-Watt University of Edinburgh. Contact him at richardjwatt@gmail.com.



Tim Cull is an application architect at Thedwick and has worked in software development for 10 years as a developer, manager, and architect. His practical experience is mostly in enterprise applications for the financial industry but also includes Web applications, clean technology, and consumer software. Cull has a BS in computer science from the University of California, Davis. Contact him at feedback@thedwick.com or via his blog at www.thedwick.com/blog.

References

1. S.L. Bain, *Emergent Design: The Evolutionary Nature of Professional Software Development*, Addison-Wesley Professional, 2008.
2. C. Matts and O. Maassen, "InfoQ: 'Real Options' Underlie Agile Practices," InfoQ, 8 June 2007; www.infoq.com/articles/real-options-enhance-agility.
3. S. Ambler, "Project Initiation: Survey Results July/August 2009," Ambysoft, Aug. 2009; www.ambysoft.com/surveys/projectInitiation2009.html.
4. N. Rozanski and E. Woods, *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives*, Addison-Wesley Professional, 2005.
5. N. Malik, "Inside Architecture: Responsibility Driven Architecture," blog, 1 Dec. 2006; <http://blogs.msdn.com/nickmalik/archive/2006/12/01/responsibility-driven-architecture.aspx>.
6. S. Ambler, "Agile Architecture: Strategies for Scaling Agile Development," Agile Modeling, 2008; www.agilemodeling.com/essays/agileArchitecture.htm#ArchitectureOwner.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.



ACM/IEEE 32nd International Conference on Software Engineering

2 – 8 MAY 2010 CAPE TOWN, SOUTH AFRICA

Cape Town International Convention Centre

REGISTER NOW!



ICSE is the premier international event for software engineering. Meet software engineering researchers and practitioners from all over the world at ICSE. ICSE 2010 is held in Cape Town, South Africa, one of the world's most beautiful cities at a world-class convention center.



ICSE includes:

- Technical Conference Program
- Software Engineering in Practice Track
- Education Track
- 18 Tutorials
- 24 Workshops
- and much more

www.sbs.co.za/icse2010